

Analiza SOLID principa

1. Single Responsibility Principle (SRP)

Princip jedne odgovornosti nalaže da svaka klasa treba imati tačno jednu odgovornost, odnosno jedan razlog za promjenu.

U ovom sistemu taj princip je jasno ispoštovan:

- Klasa **Knjiga** je zadužena isključivo za čuvanje podataka o knjizi (naziv, autor, ISBN, kategorija, status i sl.)
- Klasa **KorisnikSistema** upravlja podacima o korisniku (lični podaci, kontakt, uloga, status aktivnosti)
- Klasa **ClanskaKartica** sadrži informacije o članskoj kartici i njenoj validnosti
- Klasa **ZahtjevZaPosudbu** modelira zahtjev korisnika za posudbu knjige
- Klasa **Posudba** predstavlja konkretan proces posuđivanja i vraćanja knjige
- Klasa **NabavkaKnjiga** upravlja procesom naručivanja knjiga
- Klasa **Dobavljac** sadrži informacije o dobavljaču

Posebno je važno naglasiti da su statusi izdvojeni u posebne klase tipa enumeracije:

- **StatusKnjige**
- **StatusZahtjeva**
- **StatusPosudbe**
- **StatusNabavke**
- **UlogaKorisnika**

Ove klase imaju isključivo jednu odgovornost – definisanje mogućih stanja sistema, čime se dodatno izbjegava preopterećenje glavnih klasa.

Na ovaj način svaka klasa ima jasno definisanu ulogu i nema miješanja odgovornosti.

2. Open/Closed Principle (OCP)

Open/Closed princip nalaže da sistem treba biti otvoren za proširenje, ali zatvoren za izmjene.

U datom modelu ovaj princip je zadovoljen na više načina:

- Statusi su implementirani kao enumeracije, što omogućava dodavanje novih vrijednosti (npr. novi status posudbe) bez izmjene postojećih klasa.
- Postojeće klase nije potrebno mijenjati kada se uvode nove funkcionalnosti, već se sistem može proširivati dodavanjem novih klasa ili proširenjem postojećih struktura.

- Model sloj je odvojen od poslovne logike, što znači da se nova pravila i funkcionalnosti mogu implementirati kroz dodatne servise bez izmjene modela.

3. Liskov Substitution Principle (LSP)

Liskov princip zamjene nalaže da objekti podtipova moraju biti zamjenjivi objektima baznog tipa bez narušavanja ispravnosti programa.

Iako u ovom modelu nema izraženog nasljeđivanja između klasa, princip je implicitno zadovoljen:

- Sve klase se koriste konzistentno kroz sistem bez neočekivanog ponašanja.
- Enumeracije poput **StatusKnjige**, **StatusPosudbe** i **StatusZahtjeva** mogu se koristiti u svim kontekstima gdje se očekuje njihov tip.
- Objekti klasa kao što su **Knjiga** i **KorisnikSistema** mogu se bez problema koristiti u različitim dijelovima sistema.

Ne postoje situacije u kojima bi zamjena objekta dovela do pogrešnog rada sistema.

4. Interface Segregation Principle (ISP)

Princip izoliranja interfejsa nalaže da klijenti ne treba da ovise o metodama koje neće upotrebljavati.

U ovom sistemu:

- Ne postoje veliki ili opšti interfejsi koji bi prisiljavali klase da implementiraju nepotrebne metode.
- Svaka klasa sadrži samo atribute koji su joj potrebni.
- Funkcionalnosti su jasno razdvojene između klasa (npr. upravljanje korisnicima, posudba i nabavka su odvojeni koncepti).

5. Dependency Inversion Principle (DIP)

Princip inverzije ovisnosti nalaže da moduli visokog nivoa ne bi trebali ovisiti od modula niskog nivoa, već oba trebaju ovisiti od apstrakcije.

U našem sistemu Dependency Inversion Principle (DIP) je ispunjen na način da su zavisnosti između klasa jednostavne, jasno definisane i ne uvode nepotrebnu složenost. Iako u implementaciji nisu korištene apstrakcije u vidu interfejsa ili apstraktnih klasa, za tim nije postojala stvarna potreba.